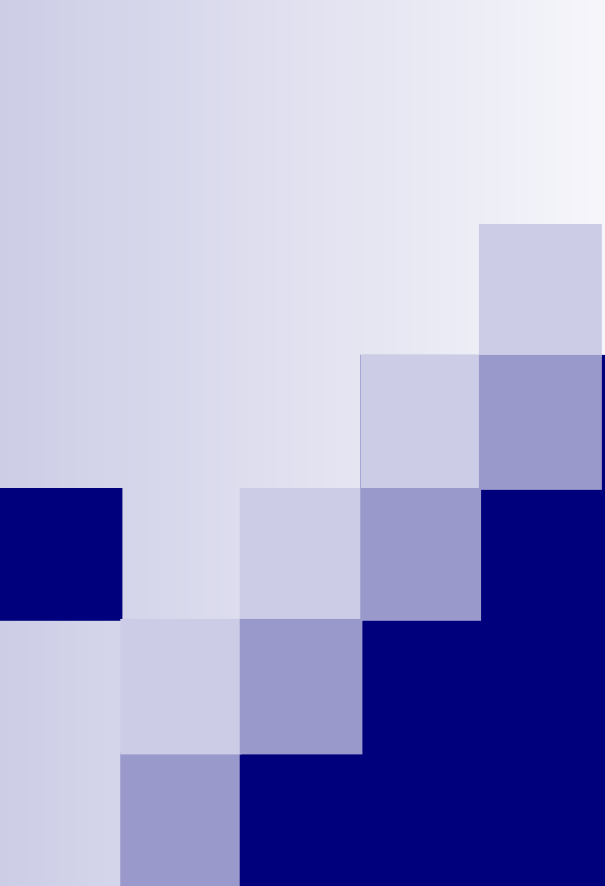




Procesadores avanzados Arquitecturas de GPU

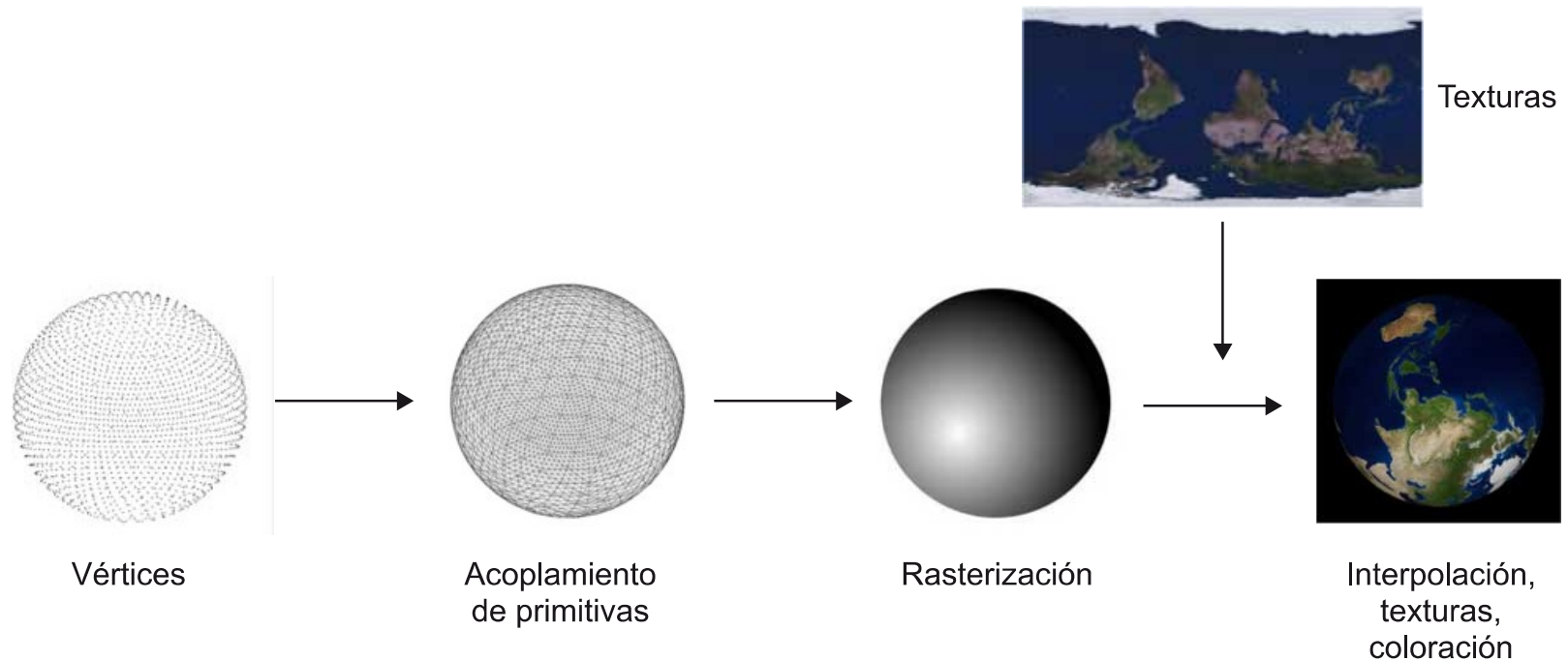
Ing. Alejandro C. Rodríguez Costello
(pubdigitalix@gmail.com)

*“Those who can imagine anything, can create the impossible.”
Alan Turing*

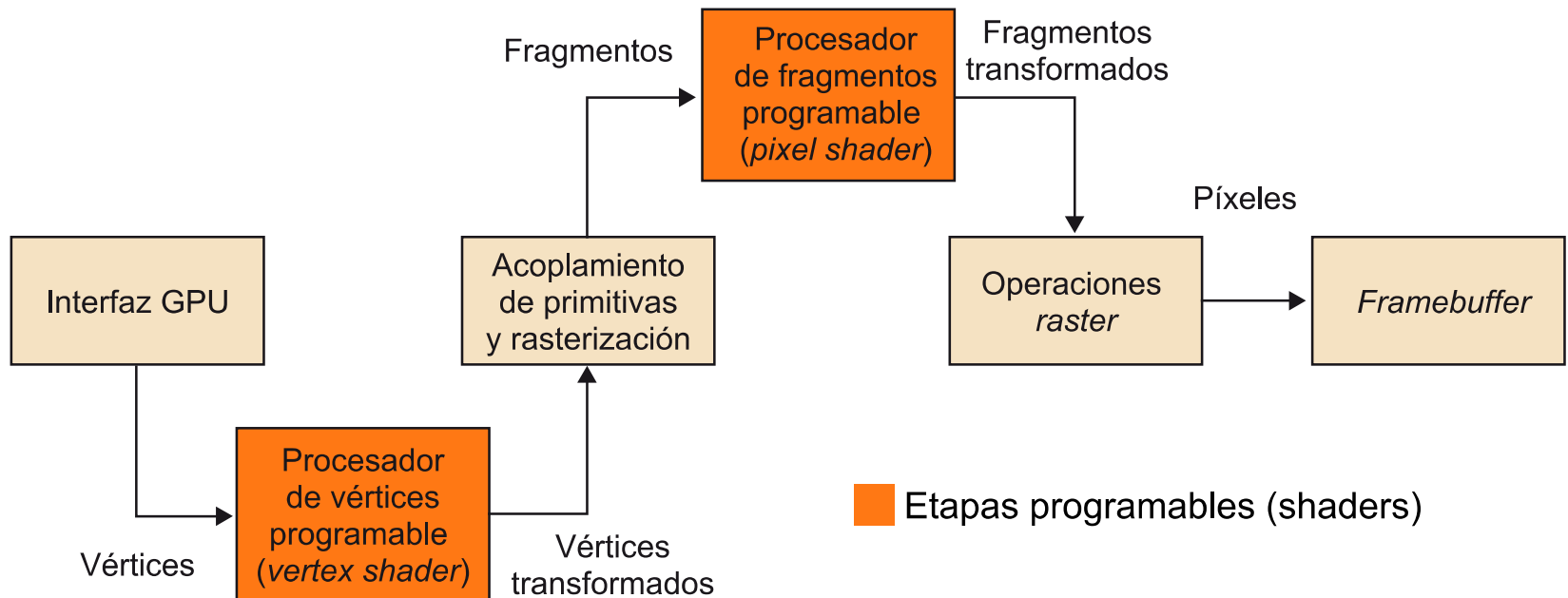
Objetivo

- Entender que es una GPU (graphic processing unit).
- Comparar las arquitecturas de CPU con GPU.
- Entender la importancia de la programación masiva paralela y las motivaciones para GPGPU (General Programing on GPU).
- Presentar un caso de uso básico: Nvidia Gforce
- Aprender conceptos básicos para programar GPUs con CUDA y OpenCL.

Pipeline gráfico conceptual



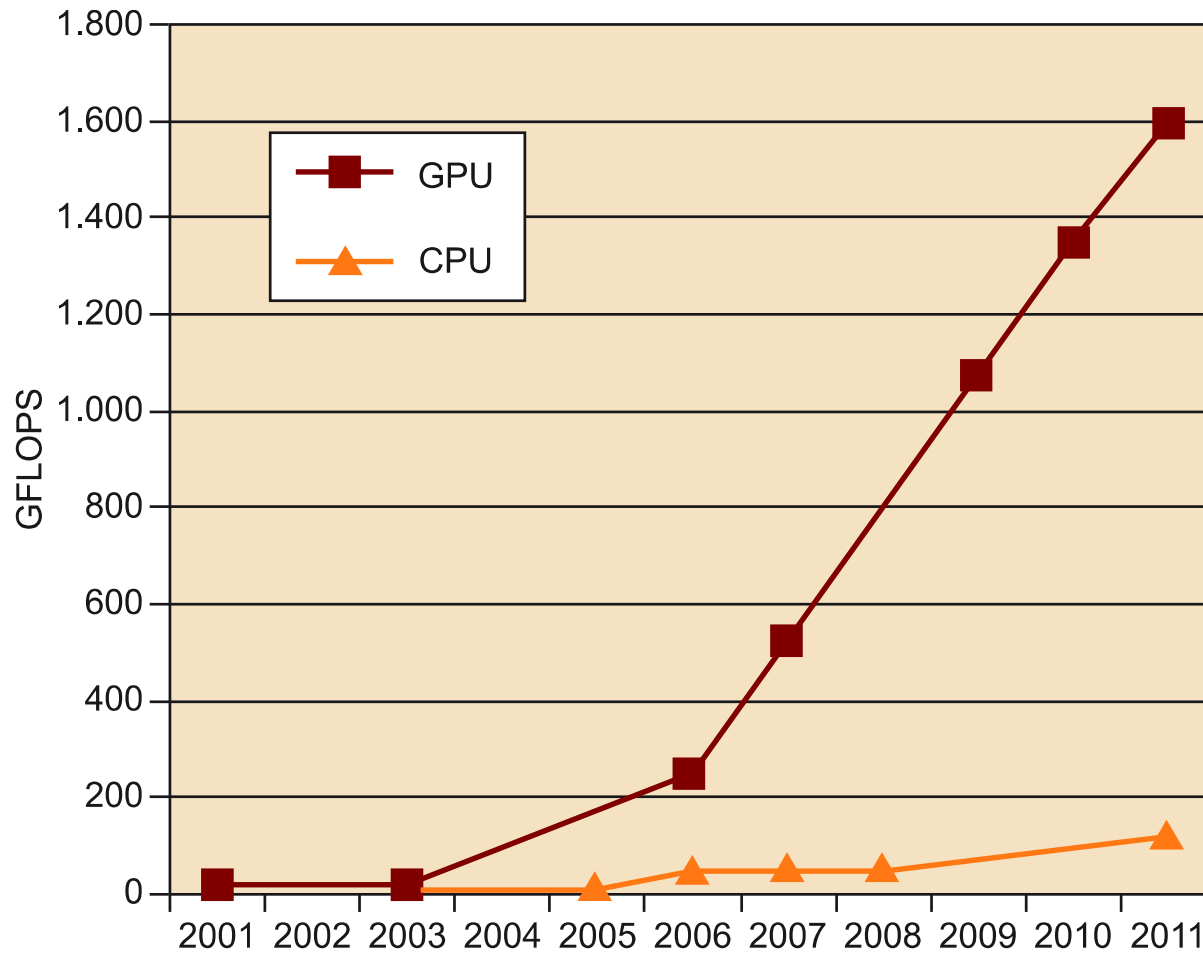
Etapas de un pipeline gráfico



Motivación para usar GPU vs CPU

- No se puede seguir aumentando el clock: debido a límites físicos de los materiales actuales.
- Límites de integración: no se pueden hacer transistores más pequeños por problemas de fabricación.
- Restricciones de energía y calor: límites tecnológicos (CMOS) y elevado consumo.
- Límites al paralelismo ILP: pipelines más largos terminan con menor rendimiento.
- Límite a manycore: no se pueden integrar demasiados cores en una sola pastilla (128 cores AMD Epic, 2019).

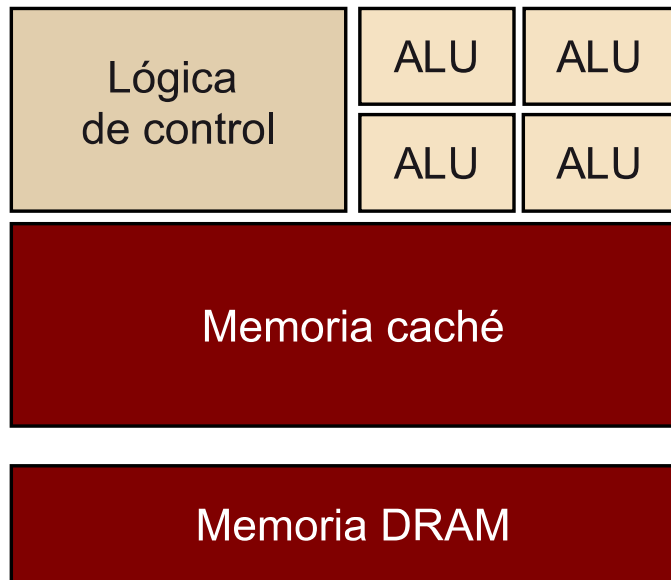
Rendimiento GPU vs CPU



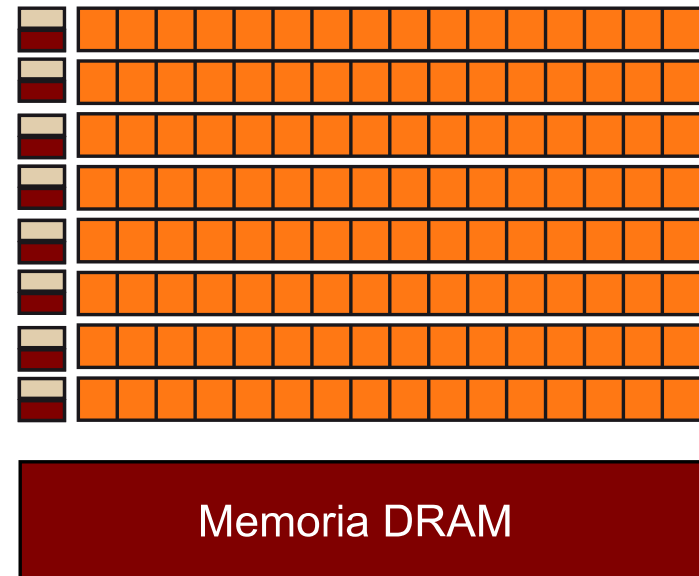
Aspectos importantes de la GPU

- Poseen un pipeline gráfico.
- Utilizan unidades aritméticas programables (shaders).
- Fueron pensadas desde el inicio SIMD.
- Soportan muchos flujos de ejecución (threads) por eso también se las llama SIMT (single instruction multiple threads)
- Pueden generalizar los shaders (unified architecture).
- Tienen una jerarquía de memoria compleja.
- Aprovechan al máximo el principio de localidad.
- Gran velocidad de acceso a memoria (DDR5).

Arquitectura CPU vs GPU

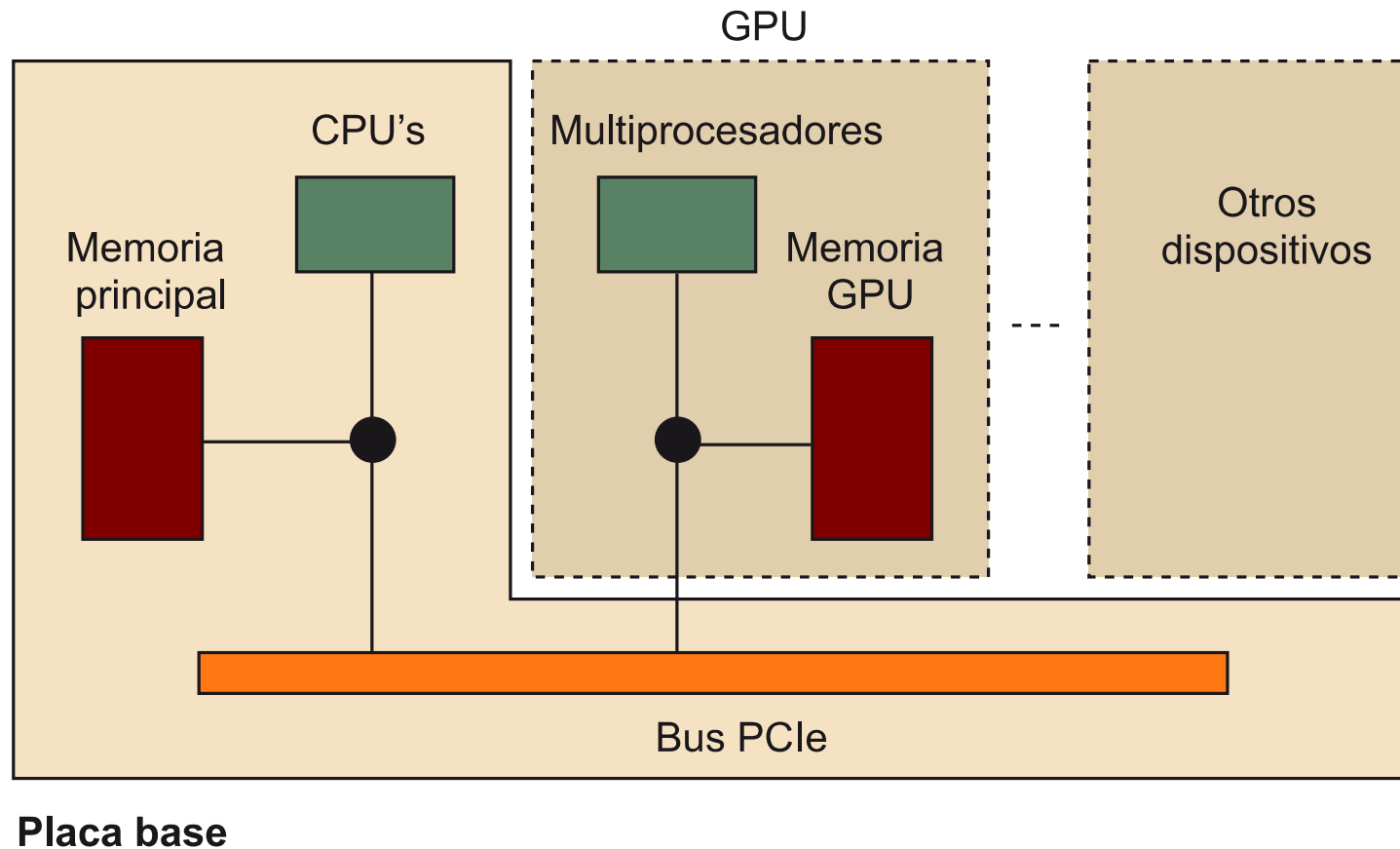


CPU



GPU

Comunicación GPU-CPU



Limitaciones de la GPU

- No puede acceder directamente a la memoria del procesador.
- Se requiere hacer **copia** explícita de los datos entre CPU y GPU (memcpy).
- No se puede hacer uso en el código de la salida estandar (printf).
- Depuración compleja y ardua.

Familia de GPUs Nvidia

- Gforce: orientada al consumo masivo multimedia (videojuegos, edición de video, fotografía digital, etc.).
- Quadro: orientada a soluciones profesionales en workstations 3D (ingeniería, arquitectura, animación).
- Tesla: orientada al computación de alta prestaciones (HPC)

Aplicaciones de la GPU

- GPGPU: usar la GPU para aplicaciones de propósito general.
- Las aplicaciones que mejor se adaptan:
 - Trabajan sobre grandes vectores de datos
 - Tienen paralelismo de grano fino SIMD
- Dominios adecuados de aplicación:
 - Álgebra lineal (producto escalar y suma de matrices)
 - Procesamiento de señales (digital image processing)
 - Mecánica computacional (computational fluidic dynamics).
 - Inteligencia artificial

Desafios para programar GPU

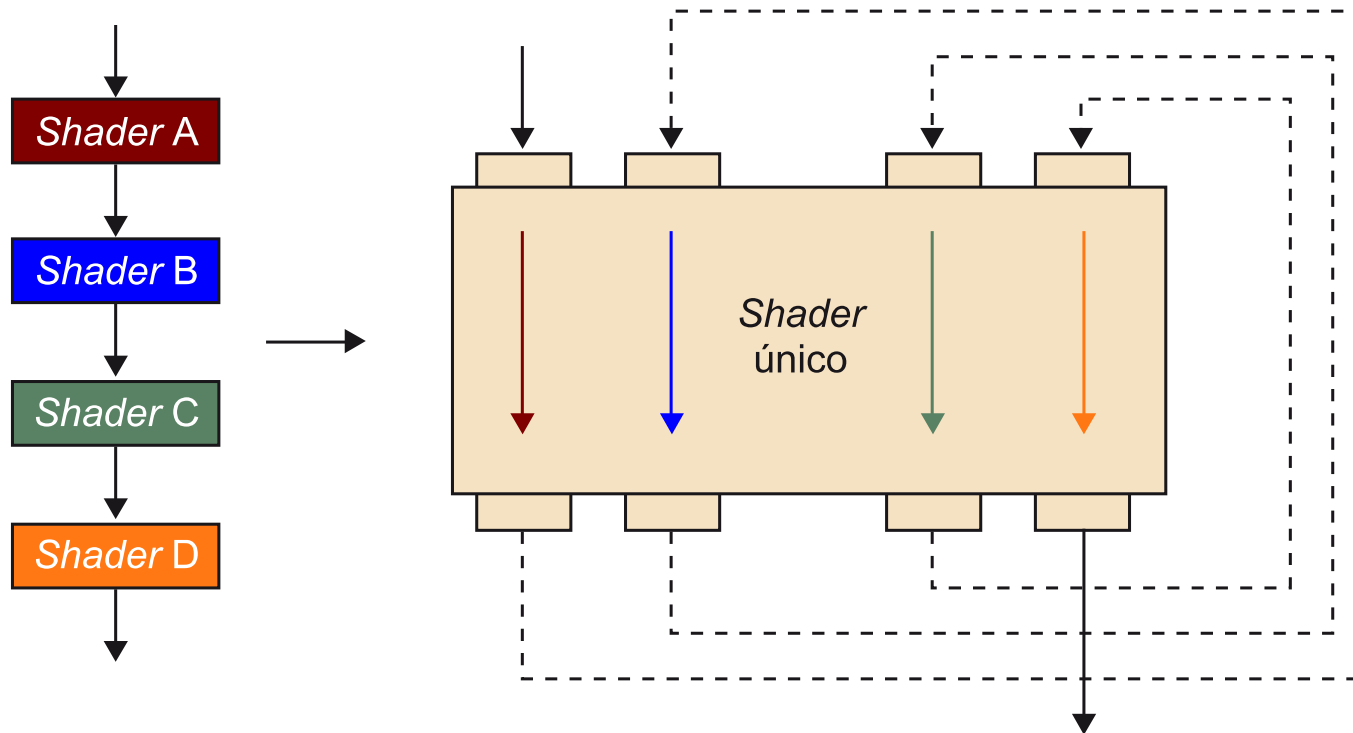
- La programación CPU y GPU no son compatibles.
- Todo dato a procesar debe convertirse en texturas para aprovechar el procesamiento por shaders.
- Se usa la técnica **render-to-texture** para escribir ya que no se puede acceder trivialmente a la memoria.
- El cálculo se realiza mediante flujos de procesador de fragmentos (pixel shader threads).
- Se hacen necesarios estandars como CUDA u OpenCL.

Caso Nvidia G80

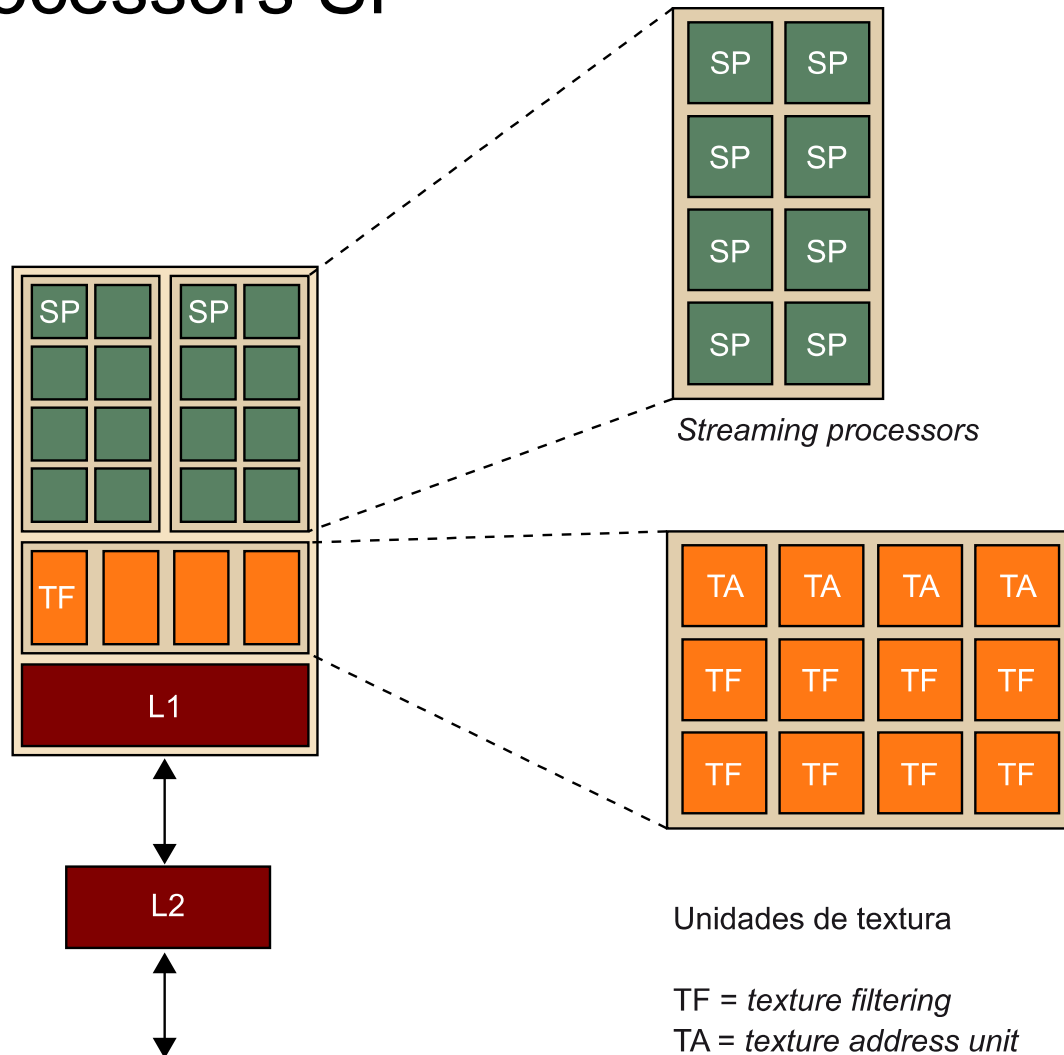
- 2006 aparece la arquitectura G80 (Tesla).
- Utiliza shaders unificados: no diferencia entre vectors o pixel shaders a nivel procesadores (SP).
- Se orienta a flujos masivos (massive threads).
- Utiliza IEEE 754 (floats).
- Pipeline gráfico con soporta DirectX 10 y OpenGL 3.3
- Utiliza dos etapas: carga y control de flujos.
- Los flujos se diferencian por su clasificación.
- El procesamiento se realiza en bloques de 16 SPs
- Comparten cache y texturas.

https://en.wikipedia.org/wiki/GeForce_8_series

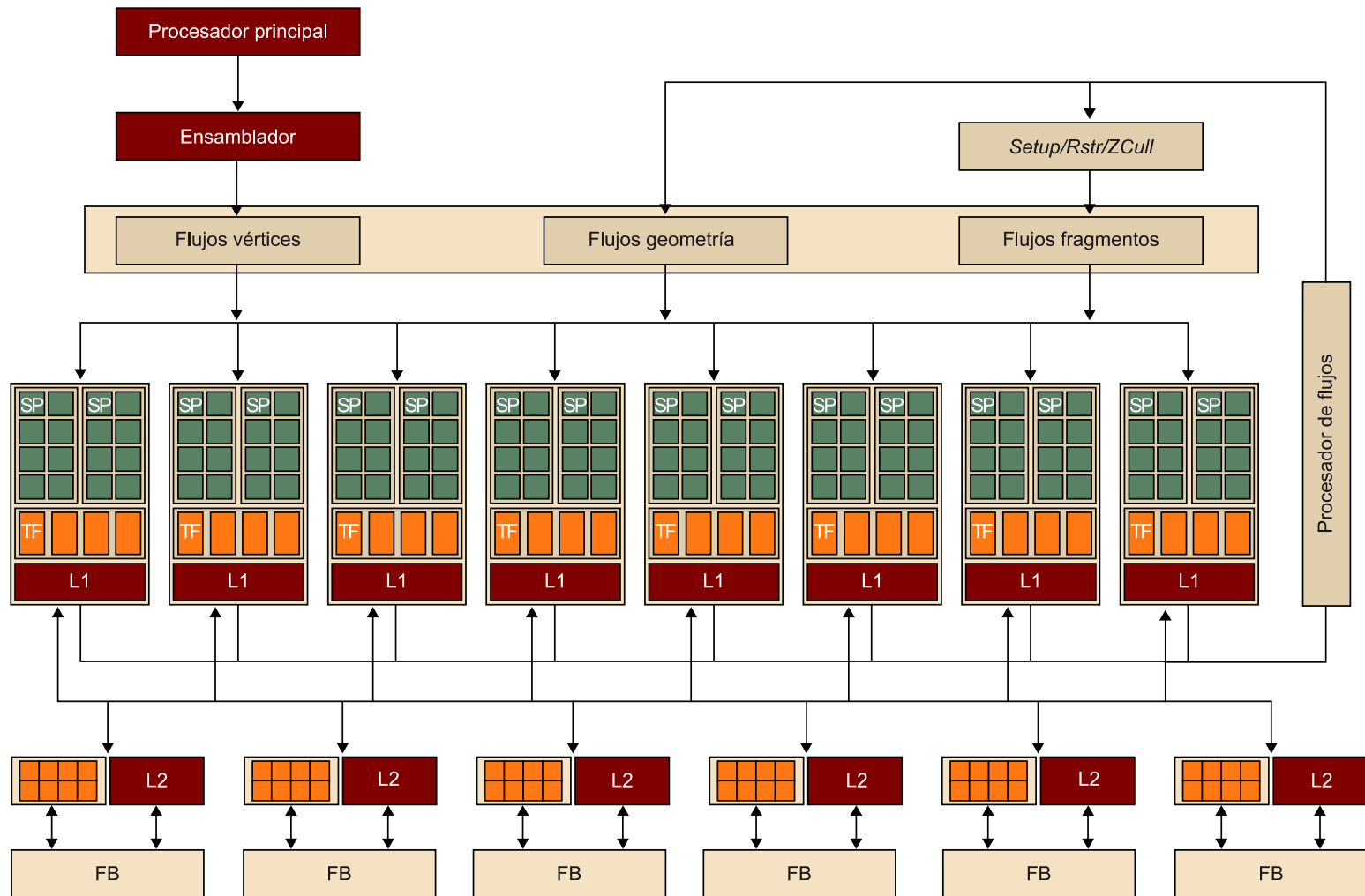
Uso de shaders unificados



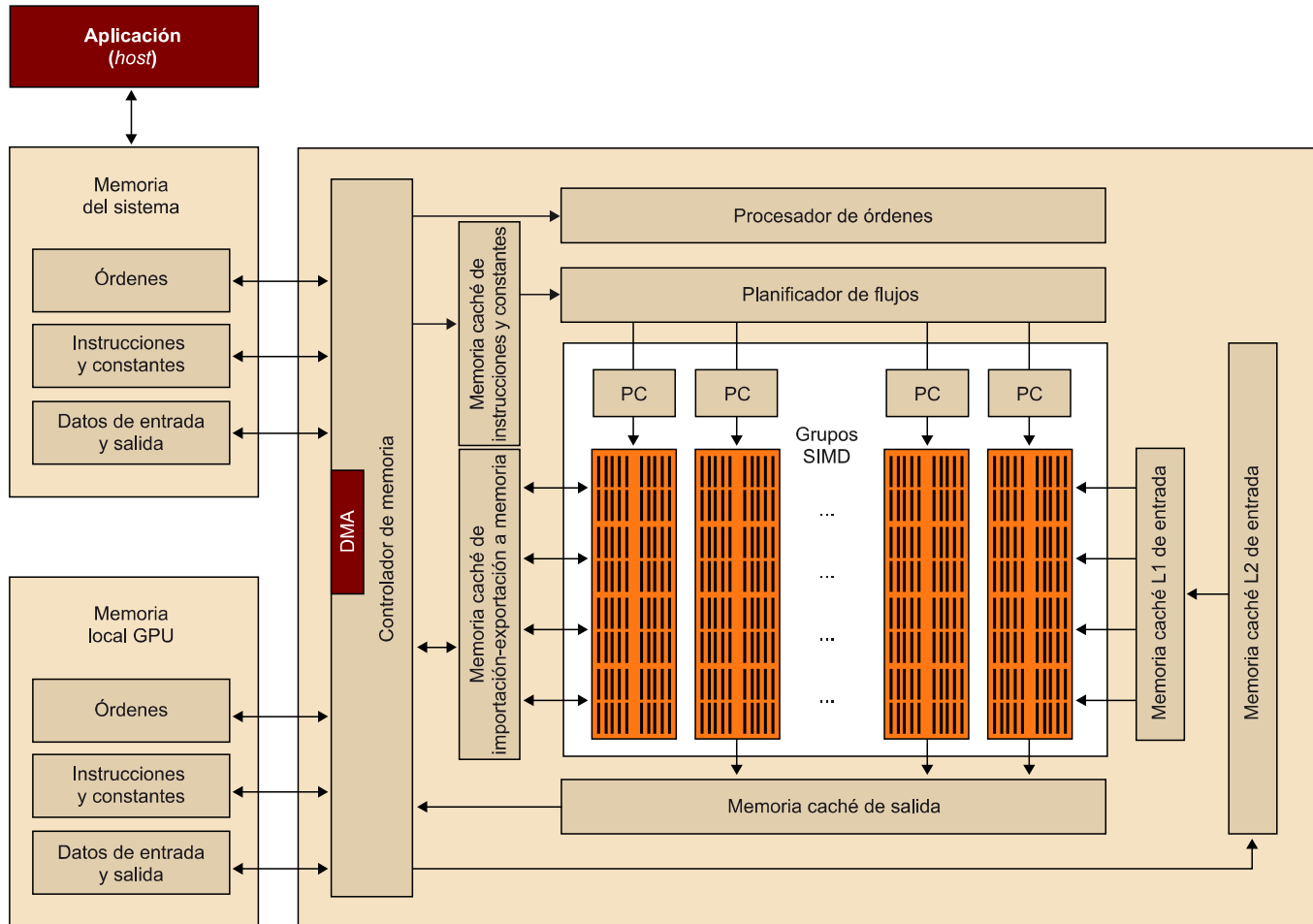
Stream processors SP



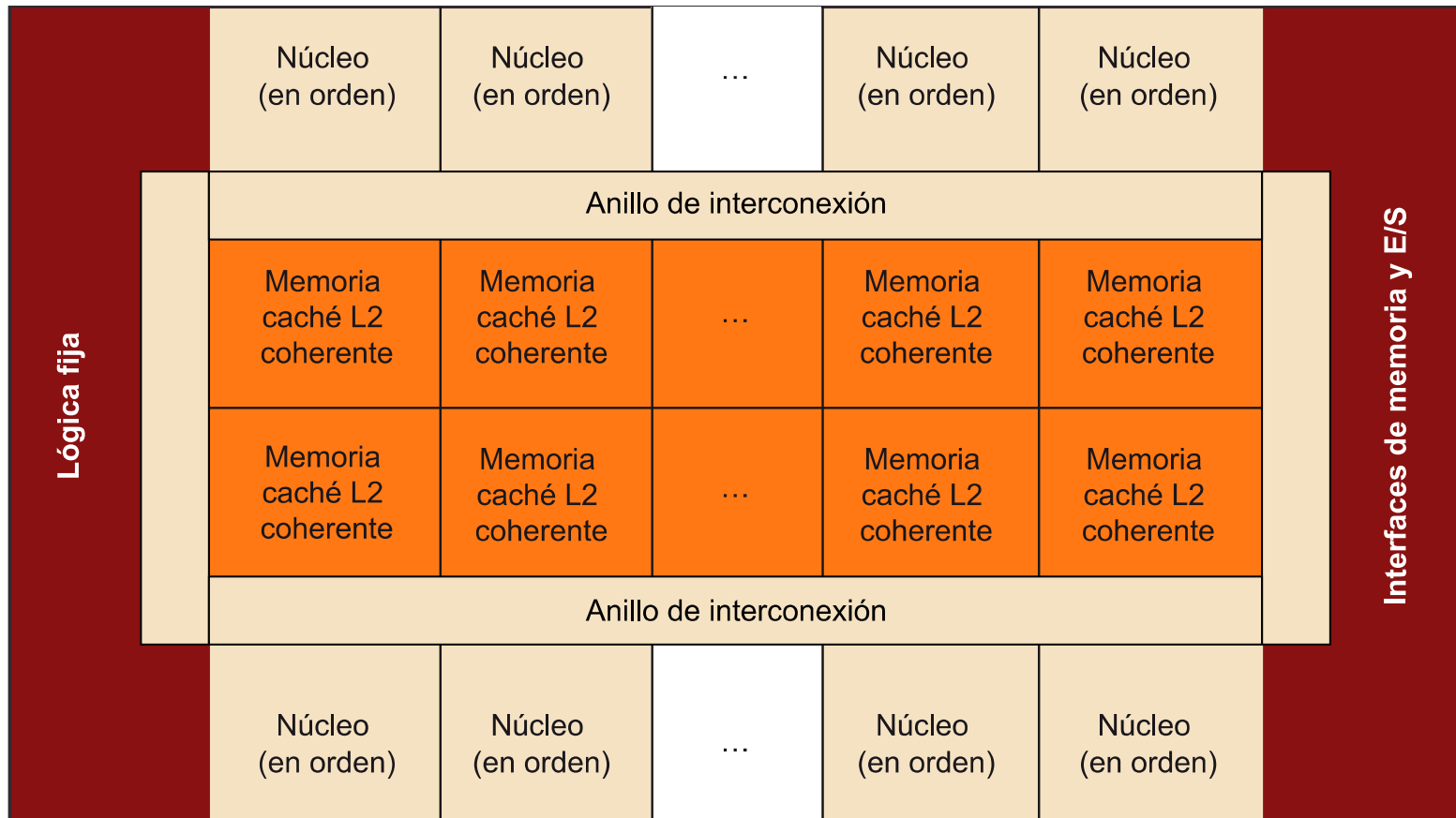
Nvidia G80 Unified architecture



Otras alternativas AMD R600



Otras alternativas Intel Larrabee (x86)

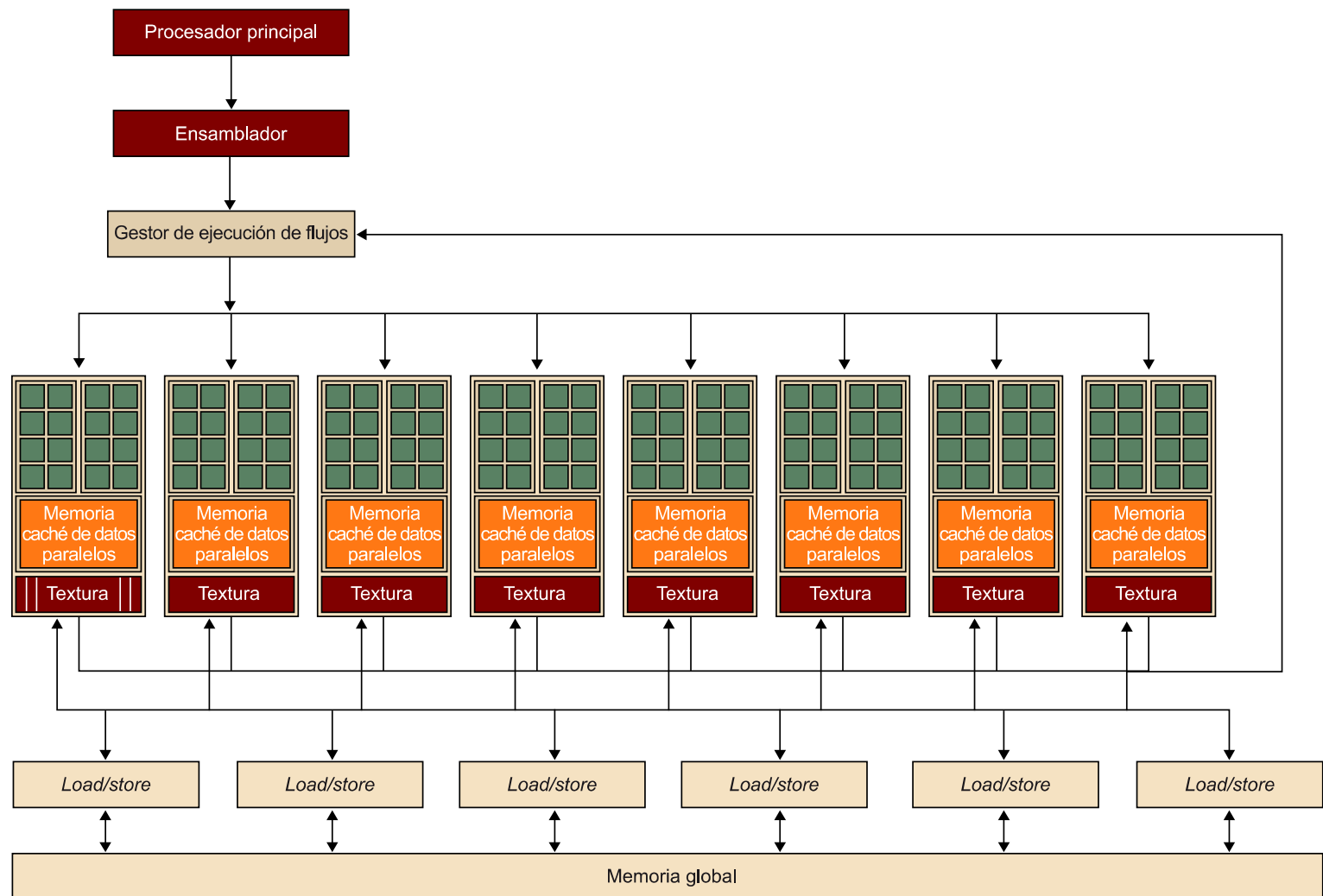


La cantidad de núcleos y la cantidad y tipo de coprocesadores y bloques de E/S son dependientes de la implementación.

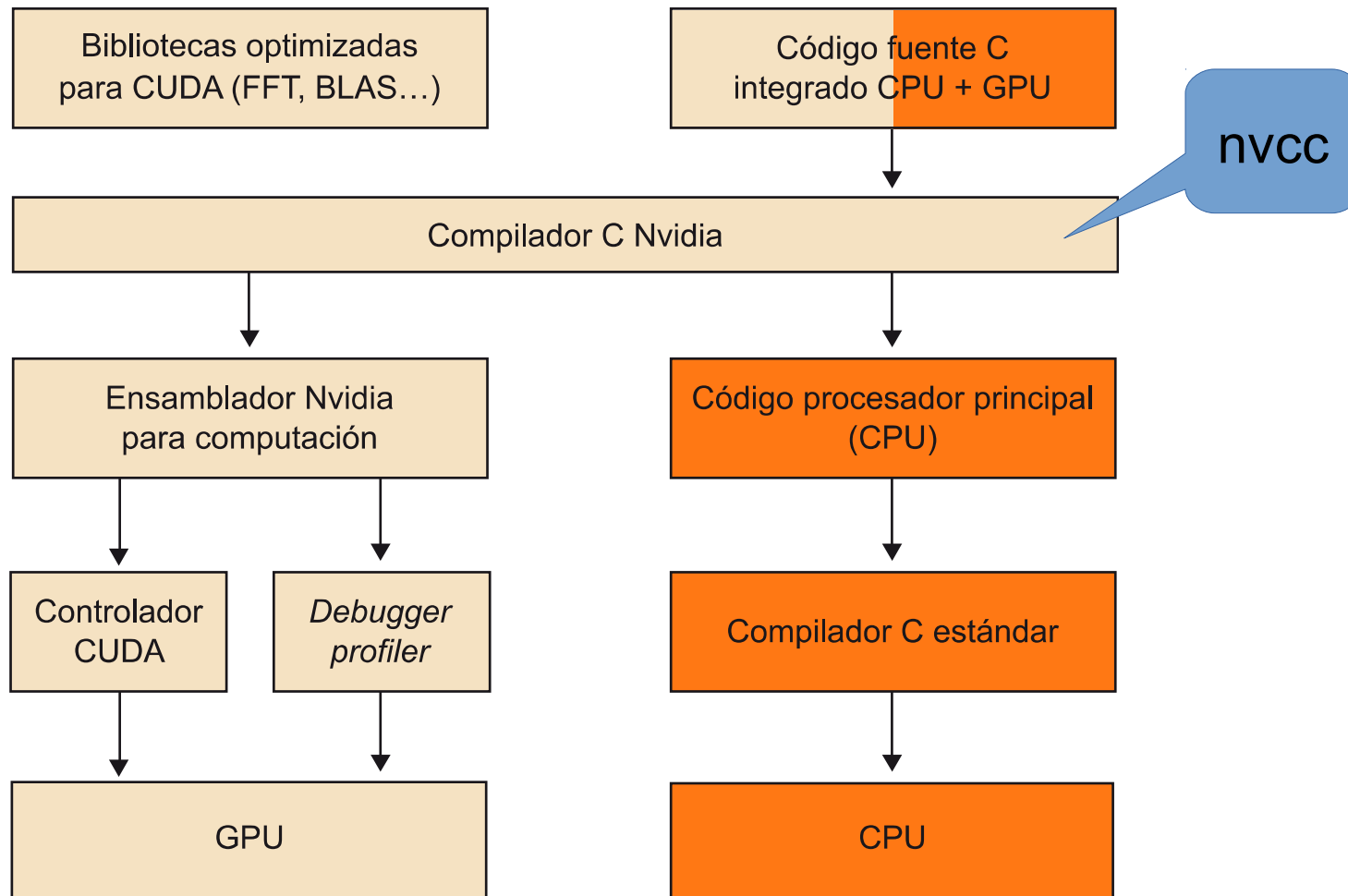
Modelos de programación GPGPU

- OpenGL: es un modelo open source para programación gráfica.
- Direct3D: es otro modelo de programación gráfica.
- CUDA: Compute Unified Device Architecture, es un modelo propietario para programación general de las GPUs de Nvidia.
- OpenCL: es un modelo open source multiplataforma para programación paralela, subconjunto de C99.

Arquitectura conceptual CUDA



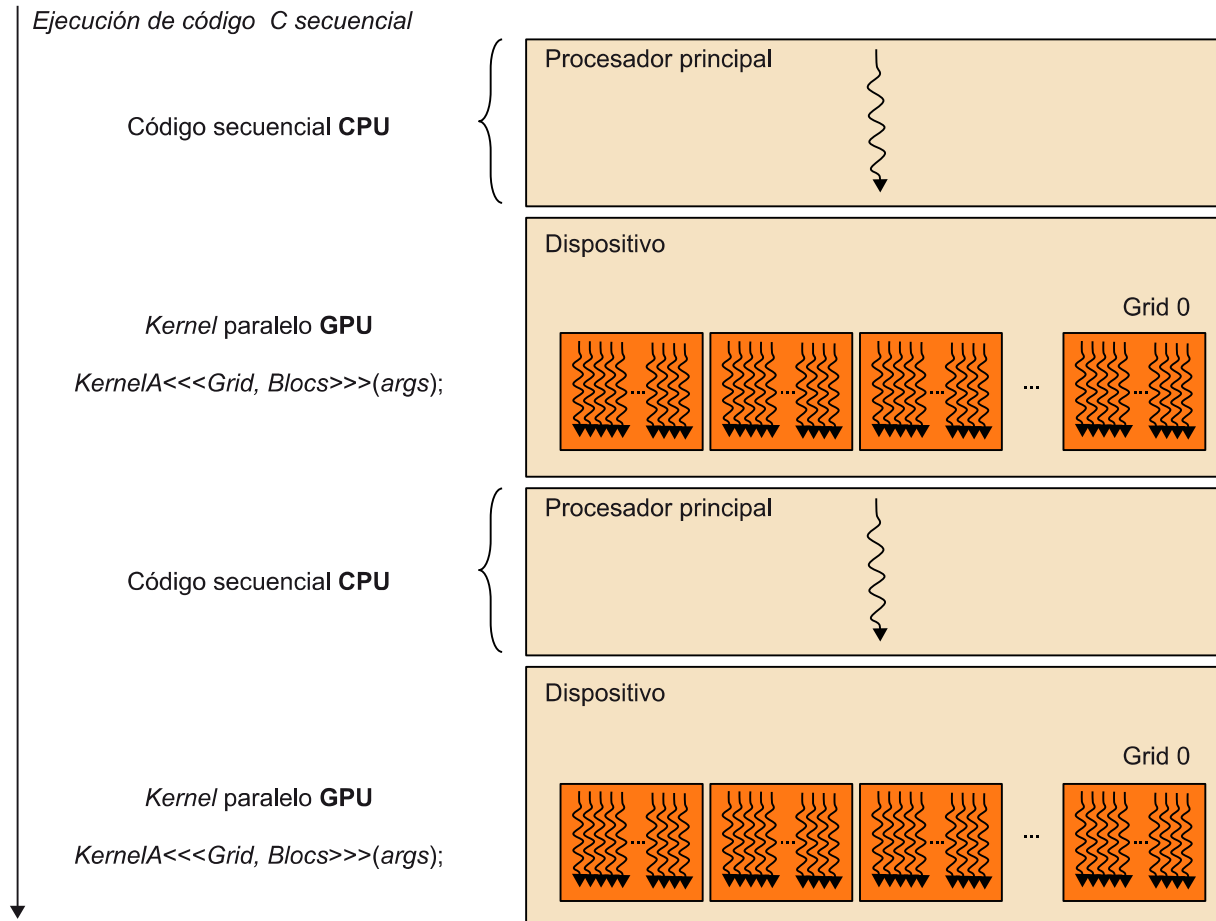
Entorno de programación CUDA



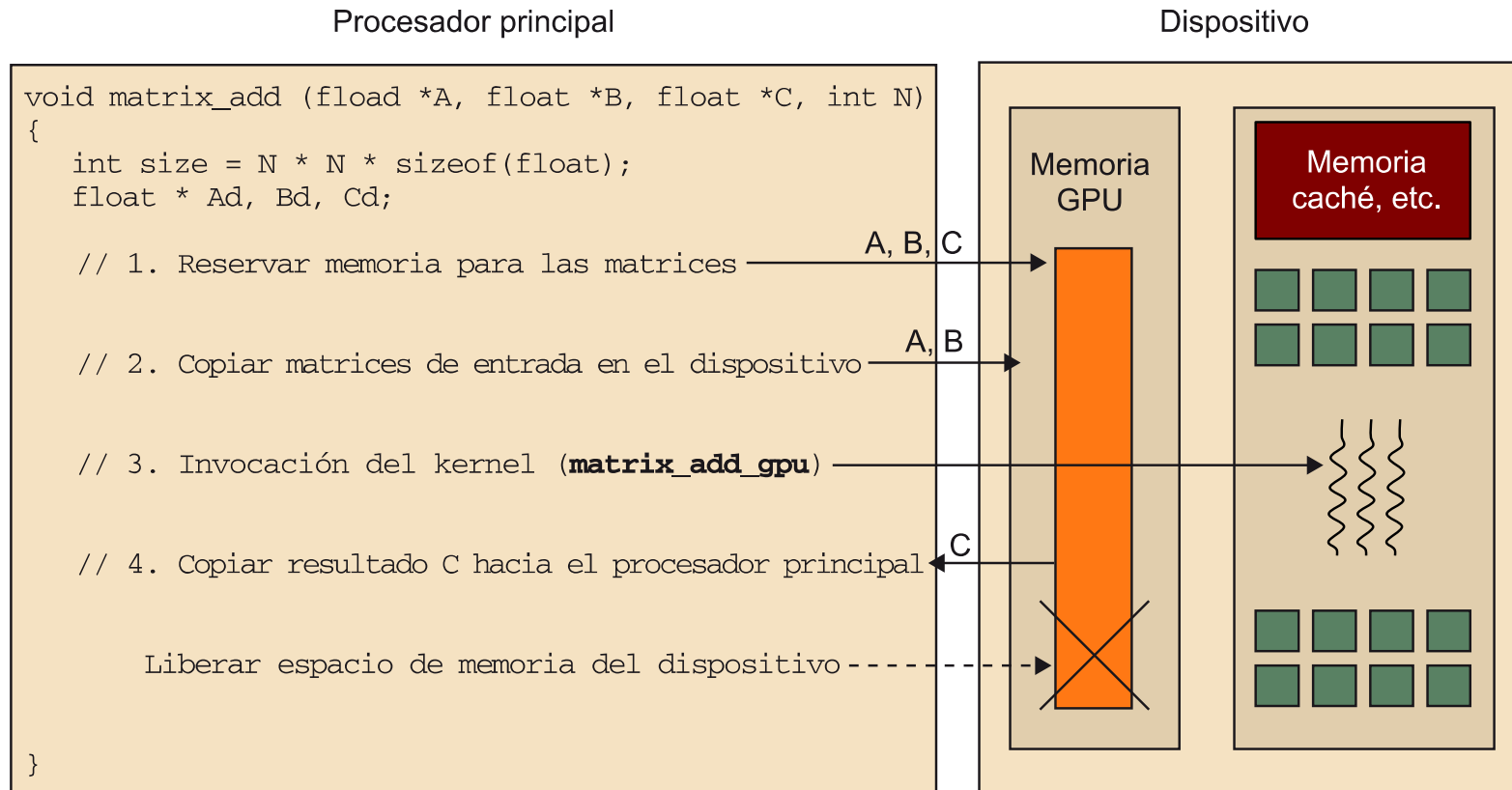
Terminología CUDA

- Kernels: funciones especiales en ANSI C. Usan palabras claves para poder tratar con el paralelismo.
- Threads: o flujos, son la unidad de ejecución básica de un kernel.
- Grids: grupos de ejecución de threads pertenecientes a un kernel.
- Block: subconjunto de ejecución de threads dentro de un grid.

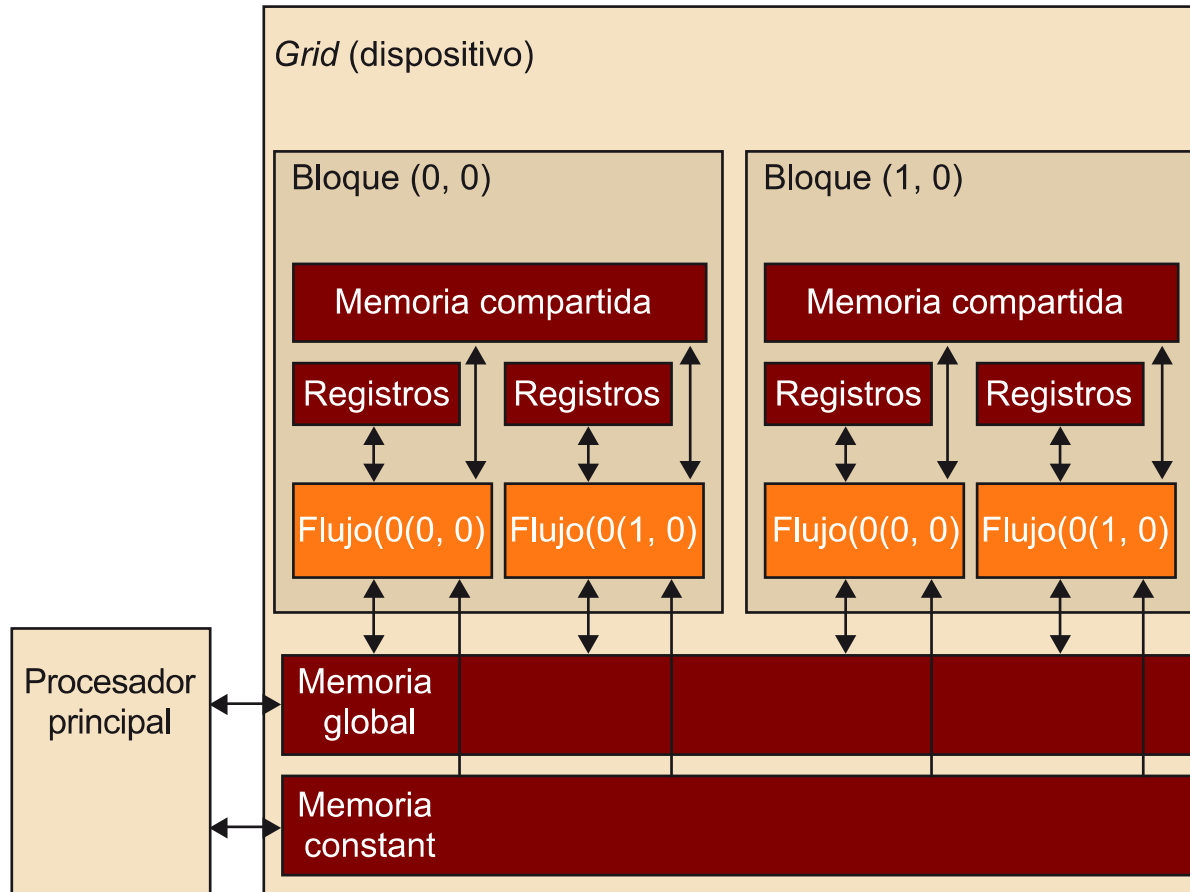
Ejemplo de ejecución de un programa CUDA



Ejemplo de ejecución de un kernel



Modelo de memoria CUDA

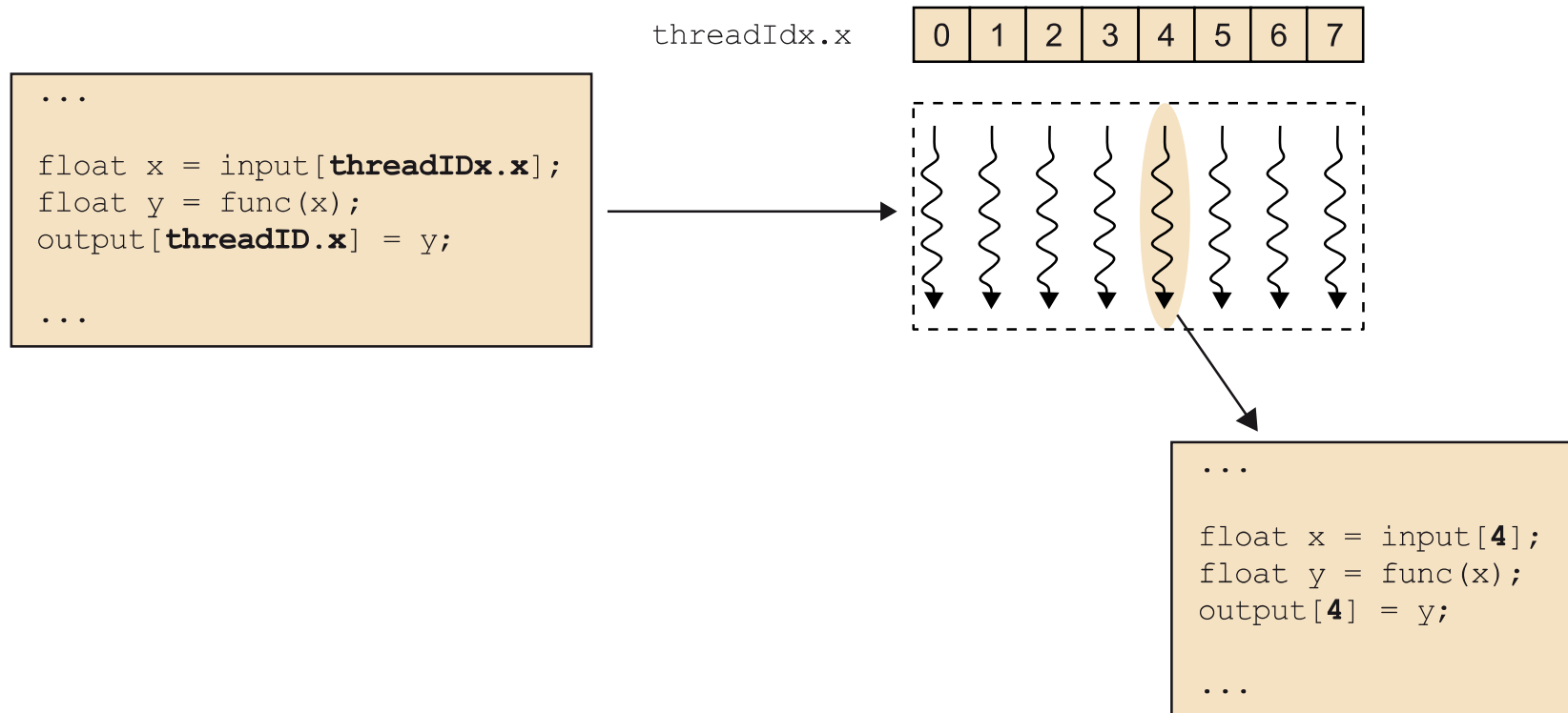


Ejemplo CPU vs kernel de CUDA

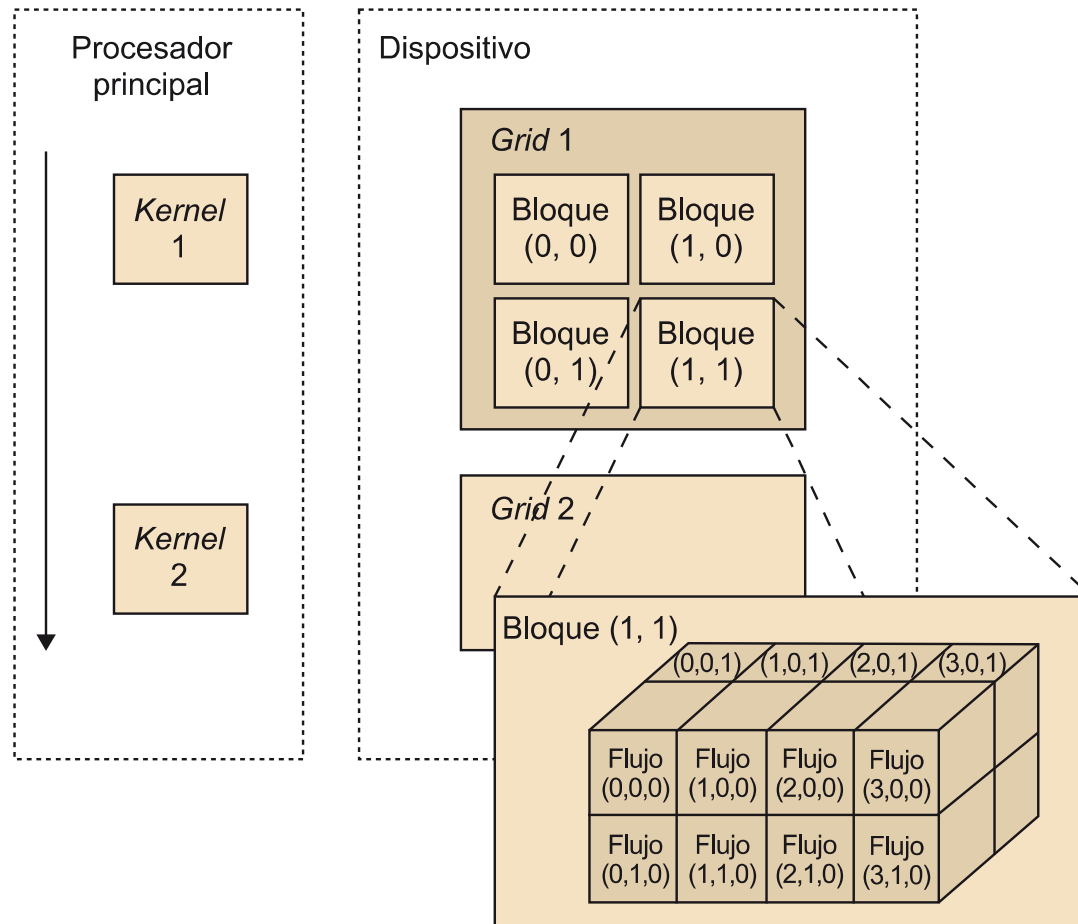
```
void matrix_add_cpu (fload *A, float *B, float *C, int N)
{
    int i, j, index;
    for (i=0; i<N; i++){
        for (j=0; j<N; j++){
            index = i+j*N;
            C[index] = C[index] + B[index];
        }
    }
}
```

```
__global__ matrix_add_gpu (fload *A, float *B, float *C, int N)
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int j = blockIdx.y * blockDim.y + threadIdx.y;
    int index = i + j*N;
    if (i<N && j<N){
        C[index] = A[index] + B[index];
    }
}
```

Identificación de un thread dentro de un kernel



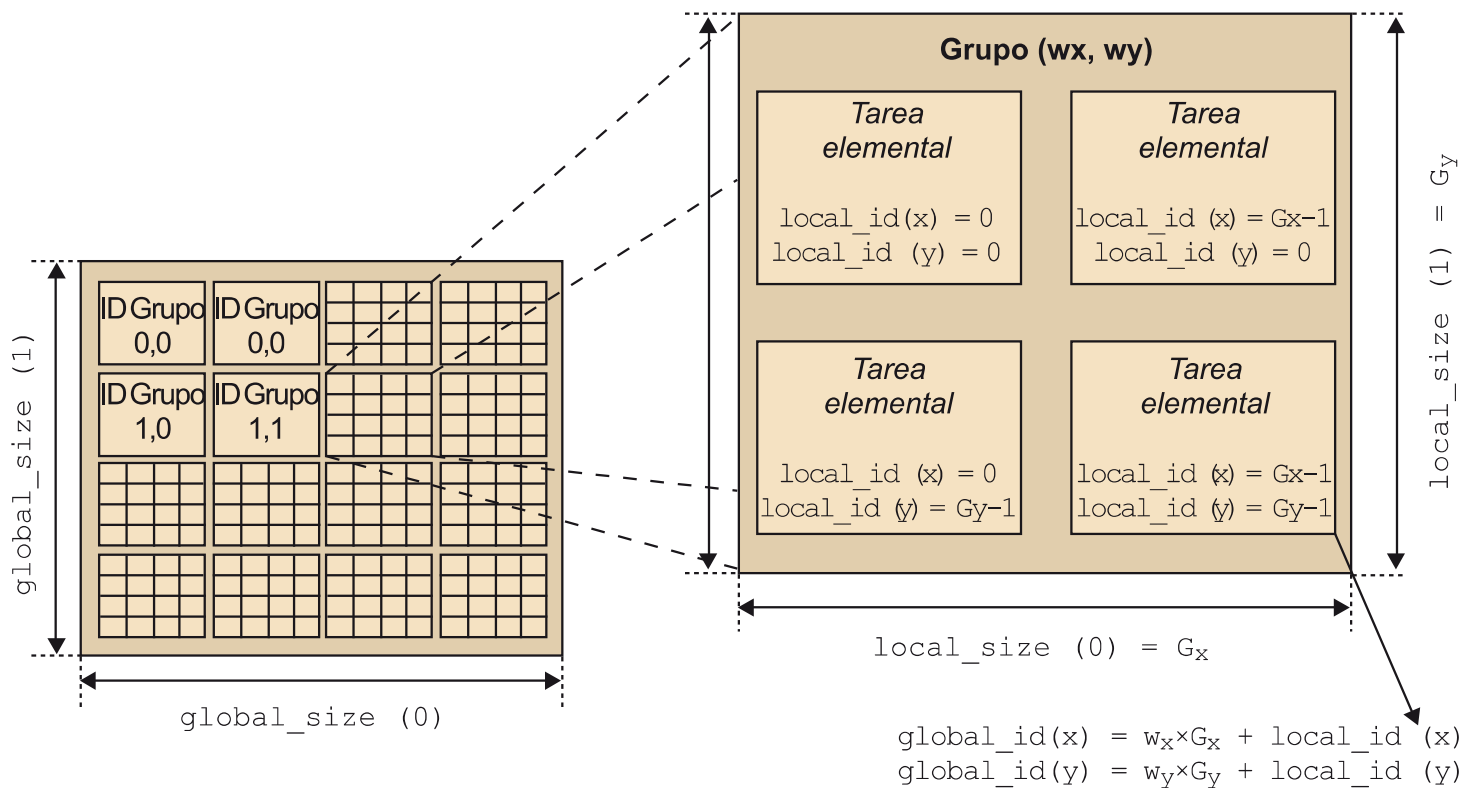
Organización de los threads dentro de un grid



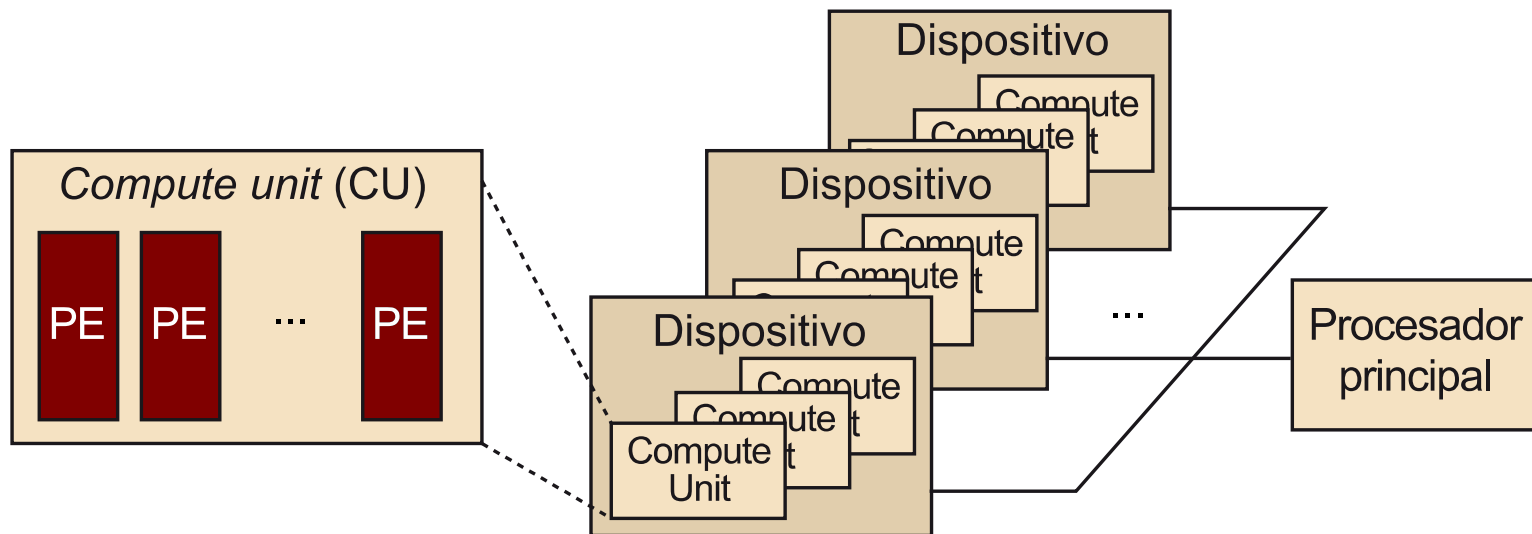
Similitudes entre OpenCL y CUDA

OpenCL	CUDA
<i>Kernel</i>	<i>Kernel</i>
Programa procesador principal	Programa procesador principal
NDRange (rango de dimensión <i>N</i>)	<i>Grid</i>
Tarea elemental (<i>work item</i>)	Flujo
Grupo de tareas (<i>work group</i>)	Bloque
<code>get_global_id(0);</code>	<code>blockIdx.x * blockDim.x + threadIdx.x</code>
<code>get_local_id(0);</code>	<code>threadIdx.x</code>
<code>get_global_size(0);</code>	<code>gridDim.x*blockDim.x</code>
<code>get_local_size(0);</code>	<code>blockDim.x</code>

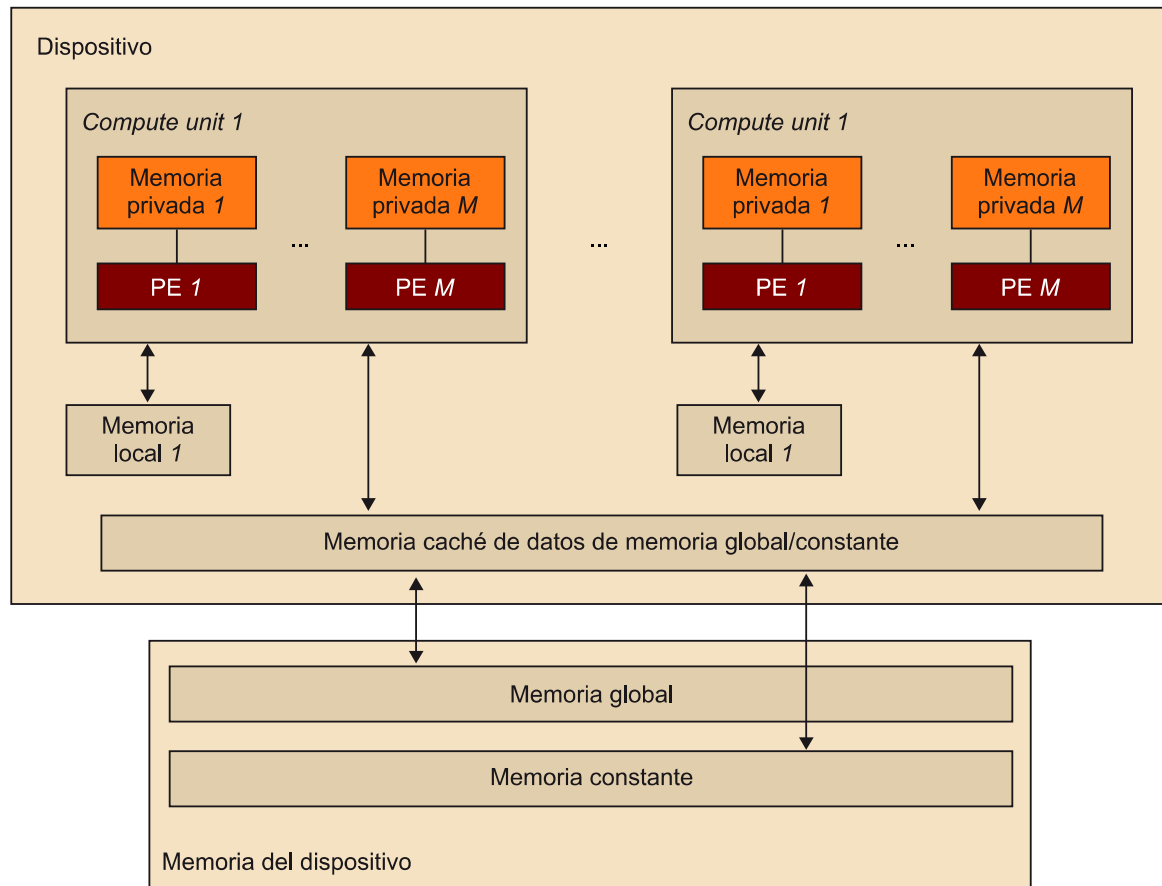
Relación entre tareas y grupos



Arquitectura conceptual OpenCL



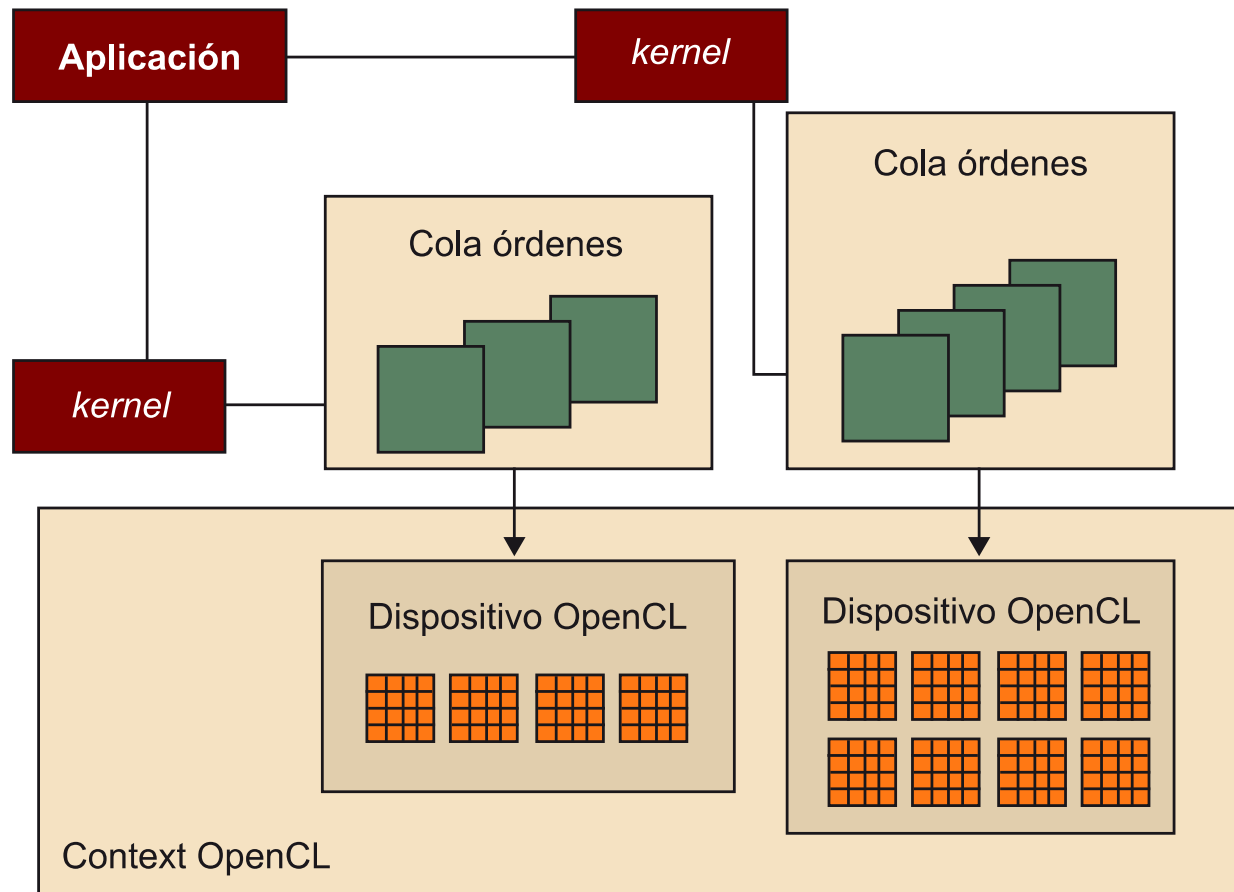
Modelo de memoria OpenCL



Ejemplo de un kernel OpenCL

```
__kernel void matrix_add_opengl ( __global const float *A,
                                   __global const float *B,
                                   __global float *C,
                                   int N) {
    int i = get_global_id(0);
    int j = get_global_id(1);
    int index = i + j*N;
    if (i<N && j<N){
        C[index] = A[index] + B[index];
    }
}
```

Gestión de dispositivos mediante contextos



Ejemplo de uso de contextos en OpenCL

```
main(){
    // Inicialización de variables, etc.
    (...)

    // 1. Creación del contexto y cola en el dispositivo
    cl_context context = clCreateContextFromType(0, CL_DEVICE_TYPE_GPU, NULL, NULL, NULL);
    // Para obtener la lista de dispositivos GPU asociados al contexto
    size_t cb;
    clGetContextInfo( context, CL_CONTEXT_DEVICES, 0, NULL, &cb);
    cl_device_id *devices = malloc(cb);
    clGetContextInfo( context, CL_CONTEXT_DEVICES, cb, devices, NULL);
    cl_cmd_queue cmd_queue = clCreateCommandQueue(context, devices[0], 0, NULL);
    // 2. Definición de los objetos en memoria (matrices A, B y C)
    cl_mem memobjs[3];
    memobjs[0] = clCreateBuffer(context, CL_MEM_READ_ONLY | CL_MEM_COPY_HOST_PTR,
                               sizeof(cl_float)*n, srcA, NULL);
    memobjs[1] = clCreateBuffer(context, CL_MEM_READ_ONLY | CL_MEM_COPY_HOST_PTR,
                               sizeof(cl_float)*n, srcB, NULL);
    memobjs[2] = clCreateBuffer(context, CL_MEM_WRITE_ONLY, sizeof(cl_float)*n, NULL, NULL);

    // 3. Definición del kernel y argumentos
    cl_program program = clCreateProgramWithSource(context, 1, &program_source, NULL, NULL);
    cl_int err = clBuildProgram(program, 0, NULL, NULL, NULL, NULL);
    cl_kernel kernel = clCreateKernel(program, "matrix_add_opencl", NULL);
    err = clSetKernelArg(kernel, 0, sizeof(cl_mem), (void *)&memobjs[0]);
    err |= clSetKernelArg(kernel, 1, sizeof(cl_mem), (void *)&memobjs[1]);
    err |= clSetKernelArg(kernel, 2, sizeof(cl_mem), (void *)&memobjs[2]);
    err |= clSetKernelArg(kernel, 3, sizeof(int), (void *)&n);

    // 4. Invocación del kernel
    size_t global_work_size[1] = n;
    err = clEnqueueNDRangeKernel(cmd_queue, kernel, 1, NULL, global_work_size,
                                NULL, 0, NULL, NULL);

    // 5. Lectura de los resultados (matriz C)
    err = clEnqueueReadBuffer(context, memobjs[2], CL_TRUE, 0, n*sizeof(cl_float),
                              dstC, 0, NULL, NULL);
    (...)
```